

Nodegent Style Guide

Project: Nodegent — CSE 115A Spring 2026

Team: Raabdulali and team

Version: 1.0

1. Code Style

Language & Framework

- TypeScript (strict mode) with React and Next.js App Router
- All new files use `.tsx` for components, `.ts` for utilities and types
- No any types — use proper typing or unknown with narrowing

Naming Conventions

Item	Convention	Example
Components	PascalCase	AssignmentCard, DailySnapshot
Hooks	camelCase with use prefix	useCanvasData, useScrollProgress
Utilities	camelCase	formatDueDate, parseCanvasToken
Constants	UPPER_SNAKE_CASE	MAX_RETRIES, API_BASE_URL
CSS classes	kebab-case	assignment-card, snapshot-header
Files (components)	PascalCase	AssignmentCard.tsx
Files (utils/hooks)	camelCase	useCanvasData.ts

File Organization

- Organize by feature, not by type
- Components co-located with their CSS and sub-components
- Maximum 800 lines per file — extract modules if exceeded

Formatting

- 2-space indentation
 - Single quotes for strings
 - Semicolons required
 - Trailing commas in multi-line objects and arrays
 - Prettier enforced on all `.ts` and `.tsx` files
-

2. Git Conventions

Branch Naming

```
feat/us-<number>-short-description  
fix/short-description  
chore/short-description
```

Commit Messages

Follow Conventional Commits:

```
feat: add Canvas sync polling interval  
fix: resolve null assignment due date crash  
chore: update dependencies  
docs: add Canvas SSO architecture notes
```

- Present tense, imperative mood
- No period at end
- Keep subject line under 72 characters

Pull Requests

- Title matches the user story or fix description
 - Include a short description of what changed and why
 - At least one team member reviews before merge
-

3. React & Component Style

- Prefer function components with hooks — no class components
- Keep components small and single-responsibility
- Props interfaces defined above the component, named <ComponentName>Props
- Avoid prop drilling beyond 2 levels — use context or lift state
- Server Components used where no client interactivity is needed

```
interface AssignmentCardProps {  
  title: string  
  dueDate: string  
  courseName: string  
}
```

```
export function AssignmentCard({ title, dueDate, courseName }:  
AssignmentCardProps) {  
  return (  
    <div className="assignment-card">  
      <h3>{title}</h3>  
      <span>{courseName}</span>  
      <span>{dueDate}</span>  
    </div>  
  )  
}
```

4. CSS & Design Tokens

- CSS custom properties for all colors, spacing, and typography
 - No hardcoded hex values or pixel sizes in components
 - Animate only compositor-friendly properties: `transform`, `opacity`, `clip-path`
 - Mobile-first responsive design — base styles for small screens, breakpoints for larger
-

5. Security Conventions

- No secrets or API keys in source code — environment variables only
 - All LLM calls made server-side — never expose keys to the browser
 - Validate and sanitize all user inputs at system boundaries
 - Canvas cookies stored server-side only, never in client storage
-

6. Testing Conventions

- E2E tests in `/e2e/` using Playwright
- Test file naming: `<feature>.spec.ts`
- Tests cover the critical happy path and one key error case per feature
- No `page.waitForTimeout` — use deterministic waits (`waitForSelector`, `waitForURL`)